```
print("hello") #updated test
```

```
hello
```

```
from google.colab import drive
drive.mount('/content/drive')

#since we don't need to extract

!cp "/content/drive/MyDrive/Kennydataset_IQ_v2.h5" /content/

'''
# getting data
!cp /content/drive/MyDrive/RadioMLDataset2018.zip /content/
!unzip -q /content/RadioMLDataset2018.zip -d /content/data/
'''
```

```
Mounted at /content/drive
'\n# getting data\n!cp /content/drive/MyDrive/RadioMLDatas
```

```
import h5py
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
import matplotlib.pyplot as plt

import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report


device = torch.device("cuda" if torch.cuda.is_available() else "cpu"
print(f"using device - {device}")

from torch.amp import autocast, GradScaler
scaler = GradScaler(device = 'cuda')
```

```
using device - cuda
```

```
import os

save_dir = "/content/drive/MyDrive/RadioML_Models"
os.makedirs(save_dir, exist_ok=True)

model_save_path = os.path.join(save_dir, "RawIQConfidence.pth")

# get rid previous checkpoint so training starts fresh if needed
if os.path.exists(model_save_path):
    os.remove(model_save_path)
    print("Removed old checkpoint. Starting fresh training.")
```

```
Removed old checkpoint. Starting fresh training.
```

```
print(os.cpu_count())
```

```
12
```

```
#!df -h /dev/shm
```

Loading the HDF5 Dataset

The dataset is stored in an HDF5 file, which is a common format for storing large amounts of numerical data. We'll use the `h5py` library to open this file and load the IQ samples (X), one-hot encoded class labels (Y), and Signal-to-Noise Ratio (SNR) values (Z).

```
#!find /content/drive/MyDrive -name "*.h5"
```

```
file_path = "/content/drive/MyDrive/Kennydataset_IQ_v2.h5"

hdf5_file = h5py.File(file_path, 'r')

X = hdf5_file["X_iq"] # - IQ samples
Y = hdf5_file["Y"] # - one hot class labels
Z = hdf5_file["Z"] # - SNR values
```

```
print(f"X shape: {X.shape}")   # signals
print(f"Y shape: {Y.shape}")   # labels
print(f"Z shape: {Z.shape}")   # SNR
print(f"SNR range: {np.unique(Z)}")
print(f"Number of classes: {Y.shape[1]}")
```

```
X shape: (346112, 4096, 2)
Y shape: (346112, 13)
Z shape: (346112, 1)
SNR range: [-20. -18. -16. -14. -12. -10.  -8.  -6.  -4.  -2.   0.
   8.  10.  12.  14.  16.  18.  20.  22.  24.  26.  28.  30.]
Number of classes: 13
```

```
# X into memory
print("Loading X into memory")
X_loaded = hdf5_file["X_iq"][:].astype(np.float32).transpose(0, 2,
print(f" X shape: {X_loaded.shape}")
```

```
Loading X into memory
 X shape: (346112, 2, 4096)
```

```
print("File attributes:")
for k, v in hdf5_file.attrs.items():
    print(k, v)
```

```
File attributes:
N_frames_per_snr [1024]
N_samples [4096]
N_snr_levels [26]
N_total [346112]
SNR_definition awgn measured over the full fs band (in-band SNR is h
SNR_max [30.]
SNR_min [-20.]
USE_CFO [1]
USE_FADING [1]
USE_PHASE_OFFSET [1]
USE_RS_VAR [1]
USE_TIMING_OFFSET [1]
complete [1]
created 20-Sep-2026 09:03:11
fs_hz [200000.]
fsk2_sep_hz [25000.]
fsk4_sep_hz [6250.]
generator datasetGenerationIQ_v2.m
gfsk_BT [0.3]
gfsk_pulse_length [4]
imp_rows cfo_hz,Rs_scale,timing_offset_samples,k_factor
matlab_version 26.1.0.3312084 (R2026a) Update 4
mod_ids [ 1  2  3  4  5  6  7  8  9 10 11 12 13]
mod_index_cpfsk [0.5]
mod_index_gfsk [0.5]
mod_list BPSK,QPSK,8PSK,16QAM,64QAM,PAM4,GFSK,CPFSK,2FSK,4FSK,B-FM,A
scope full
seed [2026]
sps [8]
symbol_rate_hz [25000.]
```

```
MOD_LABELS = [
    "BPSK", "QPSK", "8PSK", "16QAM", "64QAM", "PAM4", "GFSK",
    "CPFSK", "2FSK", "4FSK", "B-FM", "AM-DSB", "AM-SSB"
]

labels = np.argmax(Y[:], axis = 1)
snrs = Z[:, 0].astype(int)

# rand seed yeeehawww
SEED = 42

torch.manual_seed(SEED)
np.random.seed(SEED)

print(f"Total samples - {len(labels):,}")
print(f"Classes       - {len(MOD_LABELS)}")
print(f"SNR levels    - {sorted(np.unique(snrs).tolist())}")
```

```
Total samples - 346,112
Classes       - 13
SNR levels    - [-20, -18, -16, -14, -12, -10, -8, -6, -4, -2, 0, 2,
```

```
print("Unique label idx:", np.unique(labels))
print("Num unique labels:", len(np.unique(labels)))

for i, mod in enumerate(MOD_LABELS):
    print(f"{i:2d}: {mod}")
```

```
Unique label idx: [ 0  1  2  3  4  5  6  7  8  9 10 11 12]
Num unique labels: 13
 0: BPSK
 1: QPSK
 2: 8PSK
 3: 16QAM
 4: 64QAM
 5: PAM4
 6: GFSK
 7: CPFSK
 8: 2FSK
 9: 4FSK
10: B-FM
11: AM-DSB
12: AM-SSB
```

```
# Cell 4 - Dataset Class (index-based, no copying)

class RadioMLDataset(Dataset):
    def __init__(self, X_data, labels, snrs, indices, global_mean,
        self.X = X_data          # reference only - X_loaded stays
        self.labels = labels[indices]
        self.snrs = snrs[indices]
        self.indices = indices
        self.global_mean = global_mean
        self.global_std = global_std

    def __len__(self):
        return len(self.indices)

    def __getitem__(self, idx):
        raw = self.X[self.indices[idx]]

        # trying standardisation
        raw = (raw - self.global_mean) / (self.global_std + 1e-6)

        sample = torch.from_numpy(raw.copy())
        label = torch.tensor(self.labels[idx], dtype=torch.long)
        snr = self.snrs[idx]
        return sample, label, snr
```

```
# trying filtering for snr first - ttraining on the higher so the m
from sklearn.model_selection import train_test_split

# Keep only samples above the SNR threshold
snr_threshold = -20
valid_indices = np.where(snrs >= snr_threshold)[0]

print(f"Using {len(valid_indices):,} samples with SNR >= {snr_thres

BATCH_SIZE = 1024

# splitting for both modulation and snr

stratify_labels = np.array([
    f"{labels[i]}_{snrs[i]}"
    for i in valid_indices
])

train_idx, temp_idx = train_test_split(
    valid_indices,
    test_size=0.30,
    random_state=SEED,
    shuffle=True,
    stratify=stratify_labels
)
```

```
temp_strat = np.array([
    f"{labels[i]}_{snrs[i]}"
    for i in temp_idx
])

val_idx, test_idx = train_test_split(
    temp_idx,
    test_size=0.50,
    random_state=SEED,
    shuffle=True,
    stratify=temp_strat
)

train_samples = X_loaded[train_idx]
global_mean = train_samples.mean()
global_std  = train_samples.std()
print(f"global_mean={global_mean:.6f}  global_std={global_std:.6f}"

train_data = RadioMLDataset(X_loaded, labels, snrs, train_idx, glob
val_data = RadioMLDataset(X_loaded, labels, snrs, val_idx, global_m
test_data = RadioMLDataset(X_loaded, labels, snrs, test_idx, global


# Creating data loaders to handle the batching, and multiprocessing
# when you try dataloader - try with pin memrory true to speed up t
train_loader = DataLoader(train_data, batch_size = BATCH_SIZE, shuf
val_loader = DataLoader(val_data, batch_size = BATCH_SIZE, shuffle
test_loader = DataLoader(test_data, batch_size = BATCH_SIZE, shuffl

print(f"Train samples : {len(train_data):,}")
print(f"Val samples   : {len(val_data):,}")
print(f"Test samples  : {len(test_data):,}")
```

```
Using 346,112 samples with SNR >= -20 dB
global_mean=-0.000041  global_std=2.389370
Train samples : 242,278
Val samples   : 51,917
Test samples  : 51,917
```

```
print(np.min(snrs[train_idx]), np.max(snrs[train_idx]))
```

```
-20 30
```

```
# write here and pretend you know what you're doing

class ResBlock(nn.Module):
  def __init__(self, channels, kernel_size = 3):
    super(ResBlock, self).__init__()
    self.conv1 = nn.Conv1d(channels, channels, kernel_size = kernel
    self.bn1 = nn.BatchNorm1d(channels)
    self.conv2 = nn.Conv1d(channels, channels, kernel_size = kernel
    self.bn2 = nn.BatchNorm1d(channels)
    self.relu = nn.ReLU()

  def forward(self, x):
    residual = x
    out = self.relu(self.bn1(self.conv1(x)))
    out = self.bn2(self.conv2(out))
    return self.relu(out + residual)

class CNNModel(nn.Module):
    def __init__(self, num_classes = 13):
        super(CNNModel, self).__init__()

        ## ADDING EXPANSAION OF CHANNELS WITH STEM
        self.stem = nn.Sequential(
            nn.Conv1d(2, 64, kernel_size = 7, padding = 3),
            nn.BatchNorm1d(64),
            nn.ReLU(),
        )

        # residual stages
        self.stage1 = nn.Sequential(ResBlock(64), ResBlock(64), nn.
        self.stage2 = nn.Sequential(nn.Conv1d(64, 128, 1), ResBlock
        self.stage3 = nn.Sequential(nn.Conv1d(128, 256, 1), ResBloc
        self.stage4 = nn.MaxPool1d(2)

        self.pool = nn.AdaptiveAvgPool1d(4) #basically keeping the


        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(1024, 256), #basically 256*4
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.stem(x)
        x = self.stage1(x)
        x = self.stage2(x)
        x = self.stage3(x)
        x = self.stage4(x)
        x = self.pool(x)
        x = self.classifier(x)
        return x
```

```
# you can tru and check the forward pass before moving onto the nex
NUM_CLASSES = 13

model = CNNModel(num_classes=NUM_CLASSES).to(device)

# Grab one batch and run it through
sample_batch, sample_labels, sample_snrs = next(iter(train_loader))
sample_batch = sample_batch.to(device)

with torch.no_grad():
    output = model(sample_batch)

print(f"Input shape  : {sample_batch.shape}")   # (64, 2, 1024)
print(f"Output shape : {output.shape}")          # (64, 24)
print(f"Model parameters: {sum(p.numel() for p in model.parameters(
```

```
Input shape  : torch.Size([1024, 2, 4096])
Output shape : torch.Size([1024, 13])
Model parameters: 1,345,741
```

```
# Setting trainig params and loss func

learning_rate = 3e-4
epochs = 200 #Run around 200 maybe for the final training.

criterion = nn.CrossEntropyLoss(label_smoothing = 0.05)
optimiser = optim.AdamW(model.parameters(), lr = learning_rate, wei
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimiser, T

print(f"Optimiser     : Adam")
print(f"Learning rate : {learning_rate}")
print(f"Loss function : CrossEntropyLoss")
print(f"Epochs        : {epochs}")
```

```
Optimiser     : Adam
Learning rate : 0.0003
Loss function : CrossEntropyLoss
Epochs        : 200
```

```
def train_model(model, train_loader, val_loader, criterion, optimis
    train_losses, val_losses = [], []
    train_accs, val_accs = [], []

    best_val_acc = 0.0
    patience = 10
    epochs_without_improvement = 0

    for epoch in range(num_epochs):

        # —— Training phase ——
        model.train()
        running_loss    = 0.0
        running_correct = 0
        running_total   = 0

        for batch_idx, (inputs, labels_batch, _) in enumerate(train
            inputs = inputs.to(device)
            labels_batch = labels_batch.to(device)


            optimiser.zero_grad()


            with autocast(device_type='cuda', dtype=torch.bfloat16)
              outputs = model(inputs)
              loss = criterion(outputs, labels_batch)

            scaler.scale(loss).backward()
            scaler.step(optimiser)
            scaler.update()

            running_loss += loss.item()

            # BECAUSE OF MIXUP REMOVING TRAINING ACC - NVM IGNORE T
            preds = outputs.argmax(dim=1)
            running_correct += (preds == labels_batch).sum().item()
            running_total += labels_batch.size(0)

            #  progress every 1000 batches
            if (batch_idx + 1) % 200 == 0:
                print(f"  Epoch {epoch+1} | Batch {batch_idx+1}/{le
                      f"| Loss: {running_loss/(batch_idx+1):.4f}")

        train_loss = running_loss / len(train_loader)
        train_acc  = running_correct / running_total

        # Validation phase

        model.eval()
        val_loss    = 0.0
        val_correct = 0
        val_total   = 0

        with torch.no_grad():
            for inputs, labels_batch, _ in val_loader:
                inputs = inputs.to(device)
                labels_batch = labels_batch.to(device)
                outputs = model(inputs)
                loss = criterion(outputs, labels_batch)

                val_loss += loss.item()
```

```
                preds = outputs.argmax(dim=1)
                val_correct += (preds == labels_batch).sum().item()
                val_total += labels_batch.size(0)

        val_loss = val_loss / len(val_loader)
        val_acc  = val_correct / val_total

        # add the scheduler too ------
        scheduler.step()

        # Testing new val loss.
        print(f"LR: {optimiser.param_groups[0]['lr']:.2e}")

        # ADDING FOR KEEPING TH E BEST RUN
        if val_acc > best_val_acc:
          best_val_acc = val_acc
          epochs_without_improvement = 0
          torch.save(model.state_dict(), model_save_path)
          print(f"Saveed best model at epoch {epoch + 1} (Val Acc:
        else:
          epochs_without_improvement += 1
          print(f"No imrpovement for {epochs_without_improvement}/{

        if epochs_without_improvement >= patience:
          print(f"\nEarly stopping at epoch {epoch+1}")
          print(f"Best validation accuracy: {best_val_acc*100:.2f}%
          break


        # Store history
        train_losses.append(train_loss)
        val_losses.append(val_loss)
        train_accs.append(train_acc)
        val_accs.append(val_acc)


        print(f"Epoch {epoch+1}/{num_epochs} | "
              f"Train Loss: {train_loss:.4f} | Train Acc: {train_ac
              f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc*100:.2


    return train_losses, val_losses, train_accs, val_accs, best_val
```

```
#### THIS IS THE MAIN TRAINING LOOP - ONLY RUN ONCE ON FPU
train_losses, val_losses, train_accs, val_accs, best_val_acc = trai
    model, train_loader, val_loader, criterion, optimiser, schedule
)
```

```
  Epoch 1 | Batch 200/237 | Loss: 1.5294
LR: 3.00e-04
Saveed best model at epoch 1 (Val Acc: 64.87 %)
Epoch 1/200 | Train Loss: 1.4762 | Train Acc: 52.54% | Val Loss: 1
  Epoch 2 | Batch 200/237 | Loss: 1.1126
LR: 3.00e-04
Saveed best model at epoch 2 (Val Acc: 68.63 %)
Epoch 2/200 | Train Loss: 1.1049 | Train Acc: 66.50% | Val Loss: 1
  Epoch 3 | Batch 200/237 | Loss: 1.0442
LR: 3.00e-04
No imrpovement for 1/10 epochs.
Epoch 3/200 | Train Loss: 1.0419 | Train Acc: 68.96% | Val Loss: 1
  Epoch 4 | Batch 200/237 | Loss: 1.0108
LR: 3.00e-04
No imrpovement for 2/10 epochs.
Epoch 4/200 | Train Loss: 1.0091 | Train Acc: 70.69% | Val Loss: 1
  Epoch 5 | Batch 200/237 | Loss: 0.9839
LR: 3.00e-04
Saveed best model at epoch 5 (Val Acc: 68.84 %)
Epoch 5/200 | Train Loss: 0.9828 | Train Acc: 72.53% | Val Loss: 1
  Epoch 6 | Batch 200/237 | Loss: 0.9545
LR: 2.99e-04
Saveed best model at epoch 6 (Val Acc: 74.22 %)
Epoch 6/200 | Train Loss: 0.9516 | Train Acc: 74.61% | Val Loss: 0
  Epoch 7 | Batch 200/237 | Loss: 0.9268
LR: 2.99e-04
No imrpovement for 1/10 epochs.
Epoch 7/200 | Train Loss: 0.9248 | Train Acc: 75.71% | Val Loss: 0
  Epoch 8 | Batch 200/237 | Loss: 0.9089
LR: 2.99e-04
No imrpovement for 2/10 epochs.
Epoch 8/200 | Train Loss: 0.9085 | Train Acc: 76.50% | Val Loss: 0
  Epoch 9 | Batch 200/237 | Loss: 0.8931
LR: 2.99e-04
No imrpovement for 3/10 epochs.
Epoch 9/200 | Train Loss: 0.8925 | Train Acc: 77.09% | Val Loss: 1
  Epoch 10 | Batch 200/237 | Loss: 0.8850
LR: 2.98e-04
Saveed best model at epoch 10 (Val Acc: 75.69 %)
Epoch 10/200 | Train Loss: 0.8851 | Train Acc: 77.36% | Val Loss: (
  Epoch 11 | Batch 200/237 | Loss: 0.8736
LR: 2.98e-04
No imrpovement for 1/10 epochs.
Epoch 11/200 | Train Loss: 0.8743 | Train Acc: 77.77% | Val Loss: (
  Epoch 12 | Batch 200/237 | Loss: 0.8642
LR: 2.97e-04
No imrpovement for 2/10 epochs.
Epoch 12/200 | Train Loss: 0.8637 | Train Acc: 78.22% | Val Loss: 1
  Epoch 13 | Batch 200/237 | Loss: 0.8549
LR: 2.97e-04
Saveed best model at epoch 13 (Val Acc: 76.69 %)
Epoch 13/200 | Train Loss: 0.8547 | Train Acc: 78.52% | Val Loss: (
  Epoch 14 | Batch 200/237 | Loss: 0.8510
LR: 2.96e-04
No imrpovement for 1/10 epochs.
Epoch 14/200 | Train Loss: 0.8489 | Train Acc: 78.78% | Val Loss: 1
  Epoch 15 | Batch 200/237 | Loss: 0.8440
```

```
# just in case the loading is messed up
# Load the best model saved
model.load_state_dict(torch.load(model_save_path, map_location=devi
model.to(device)
model.eval()

print("best model mounted and loaded.")
print(f"Best validation accuracy: {best_val_acc*100:.2f}%")
```

```
best model mounted and loaded.
Best validation accuracy: 77.00%
```

```
# Cell 11 - Plot Training History

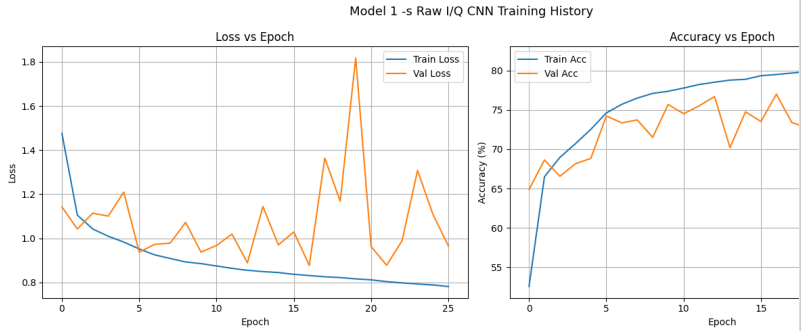
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Loss curves
ax1.plot(train_losses, label="Train Loss")
ax1.plot(val_losses,   label="Val Loss")
ax1.set_title("Loss vs Epoch")
ax1.set_xlabel("Epoch")
ax1.set_ylabel("Loss")
ax1.legend()
ax1.grid(True)

# Accuracy curves
ax2.plot([a*100 for a in train_accs], label="Train Acc")
ax2.plot([a*100 for a in val_accs],   label="Val Acc")
ax2.set_title("Accuracy vs Epoch")
ax2.set_xlabel("Epoch")
ax2.set_ylabel("Accuracy (%)")
ax2.legend()
ax2.grid(True)

plt.suptitle("Model 1 - Raw I/Q CNN Training History", fontsize=13)
plt.tight_layout()
plt.show()

# statistical scores you can go.
```



for evaluation, make sure to use macro f1 instead of f1 - because some modulation types have worse classification ability at the same low snr compared to others type shi

```
all_preds = []
all_labels = []
all_snrs = []
all_conf = []
all_true_prob = []
all_entropy = []
all_probs = []

model.eval()
with torch.no_grad():
    for inputs, labels_batch, snrs_batch in test_loader:
        inputs = inputs.to(device)
        labels_dev = labels_batch.to(device)

        outputs = model(inputs)
        probs = torch.softmax(outputs, dim = 1)

        preds = probs.argmax(dim = 1)
        conf = probs.max(dim = 1).values
        true_prob = probs[torch.arange(probs.shape[0]), labels_dev]
        entropy = -(probs * torch.log(probs + 1e-12)).sum(dim = 1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels_batch.numpy())
        all_snrs.extend(snrs_batch.numpy())
        all_conf.extend(conf.cpu().numpy())
        all_true_prob.extend(true_prob.cpu().numpy())
        all_entropy.extend(entropy.cpu().numpy())
        all_probs.append(probs.cpu().numpy())

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)
all_snrs = np.array(all_snrs)
all_conf = np.array(all_conf)
all_true_prob = np.array(all_true_prob)
all_entropy = np.array(all_entropy)
all_probs = np.concatenate(all_probs, axis = 0)

print("Done. Test samples:", len(all_preds))
```

```
Done. Test samples: 51917
```

```
# rebuilding the metrics that the new eval loop no longer computes
from sklearn.metrics import classification_report, confusion_matrix

overall_acc = accuracy_score(all_labels, all_preds)
macro_f1 = f1_score(all_labels, all_preds, average = 'macro')

print(f"Overall Accuracy: {overall_acc*100:.2f}%")
print(f"F1 Score (the macro one) : {macro_f1:.4f}")
print("\nClassification Report:")
print(classification_report(all_labels, all_preds, target_names = M

snr_range = sorted(np.unique(all_snrs).tolist())
snr_accs = {}

for snr in snr_range:
    mask = all_snrs == snr
    snr_acc = accuracy_score(all_labels[mask], all_preds[mask])
    snr_accs[snr] = snr_acc
    print(f"SNR {snr:4d} dB | Acc: {snr_acc*100:.2f}%")
```

```
Overall Accuracy: 76.90%
F1 Score (the macro one) : 0.7699

Classification Report:
              precision    recall  f1-score   support

        BPSK      0.586     0.850     0.694      3994
        QPSK      0.609     0.711     0.656      3993
        8PSK      0.757     0.662     0.706      3988
       16QAM      0.828     0.534     0.649      3994
       64QAM      0.735     0.638     0.683      3996
        PAM4      0.942     0.732     0.824      3992
        GFSK      0.907     0.814     0.858      3994
       CPFSK      0.931     0.715     0.809      3995
        2FSK      0.680     0.963     0.797      3996
        4FSK      0.803     0.787     0.795      3994
        B-FM      0.865     0.806     0.835      3992
      AM-DSB      0.690     0.966     0.805      3998
      AM-SSB      0.994     0.820     0.899      3991

    accuracy                          0.769     51917
   macro avg      0.794     0.769     0.770     51917
weighted avg      0.794     0.769     0.770     51917

SNR  -20 dB | Acc: 12.27%
SNR  -18 dB | Acc: 16.02%
SNR  -16 dB | Acc: 20.97%
SNR  -14 dB | Acc: 27.94%
SNR  -12 dB | Acc: 34.00%
SNR  -10 dB | Acc: 44.62%
SNR   -8 dB | Acc: 57.83%
SNR   -6 dB | Acc: 70.32%
SNR   -4 dB | Acc: 79.41%
SNR   -2 dB | Acc: 85.53%
SNR    0 dB | Acc: 89.07%
SNR    2 dB | Acc: 91.49%
```

```
SNR    4 dB | Acc: 95.09%
SNR    6 dB | Acc: 96.49%
SNR    8 dB | Acc: 97.75%
SNR   10 dB | Acc: 98.05%
SNR   12 dB | Acc: 98.30%
SNR   14 dB | Acc: 98.30%
SNR   16 dB | Acc: 97.94%
SNR   18 dB | Acc: 98.25%
SNR   20 dB | Acc: 98.00%
SNR   22 dB | Acc: 98.45%
SNR   24 dB | Acc: 98.50%
SNR   26 dB | Acc: 98.35%
SNR   28 dB | Acc: 98.60%
SNR   30 dB | Acc: 98.00%
```

```
correct = (all_preds == all_labels)

print(f"Mean confidence (all)     : {all_conf.mean():.3f}")
print(f"Mean confidence (correct) : {all_conf[correct].mean():.3f}"
print(f"Mean confidence (wrong)   : {all_conf[~correct].mean():.3f}
print(f"Mean prob on true class   : {all_true_prob.mean():.3f}")
print(f"Mean entropy              : {all_entropy.mean():.3f}")
print(f"  ln(6)  = {np.log(6):.3f}  (uncertain across 6 classes)")
print(f"  ln(13) = {np.log(13):.3f}  (no idea at all)")
```

```
Mean confidence (all)     : 0.759
Mean confidence (correct) : 0.889
Mean confidence (wrong)   : 0.329
Mean prob on true class   : 0.714
Mean entropy              : 0.763
  ln(6)  = 1.792  (uncertain across 6 classes)
  ln(13) = 2.565  (no idea at all)
```

```
high = (all_snrs >= 20)

print("High SNR only (>= 20 dB)")
print("Class      Acc      Conf    TrueP   Entropy")
for i in range(13):
    mask = (all_labels == i) & high
    acc_i = (all_preds[mask] == all_labels[mask]).mean()
    conf_i = all_conf[mask].mean()
    true_i = all_true_prob[mask].mean()
    ent_i = all_entropy[mask].mean()
    print(f"{MOD_LABELS[i]:8s} {acc_i*100:6.2f}%   {conf_i:.3f}   {
```

```
High SNR only (>= 20 dB)
Class      Acc      Conf    TrueP   Entropy
BPSK      99.89%   0.946   0.946   0.331
QPSK      99.13%   0.941   0.939   0.309
8PSK      99.78%   0.947   0.947   0.285
16QAM     82.07%   0.878   0.813   0.423
64QAM     97.61%   0.889   0.884   0.412
PAM4     100.00%   0.936   0.936   0.367
GFSK     100.00%   0.974   0.974   0.177
CPFSK     99.89%   0.974   0.973   0.174
2FSK     100.00%   0.971   0.971   0.200
4FSK     100.00%   0.964   0.964   0.228
B-FM     100.00%   0.966   0.966   0.224
AM-DSB    99.78%   0.958   0.958   0.253
AM-SSB    99.89%   0.977   0.976   0.160
```

```
for snr in sorted(np.unique(all_snrs)):
    mask = (all_snrs == snr)
    acc_s = (all_preds[mask] == all_labels[mask]).mean()
    conf_s = all_conf[mask].mean()
    ent_s = all_entropy[mask].mean()
    print(f"SNR {snr:4d} dB | Acc: {acc_s*100:5.2f}% | Conf: {conf_
```

```
SNR  -20 dB | Acc: 12.27% | Conf: 0.291 | Entropy: 2.158
SNR  -18 dB | Acc: 16.02% | Conf: 0.257 | Entropy: 2.193
SNR  -16 dB | Acc: 20.97% | Conf: 0.283 | Entropy: 2.083
SNR  -14 dB | Acc: 27.94% | Conf: 0.337 | Entropy: 1.908
SNR  -12 dB | Acc: 34.00% | Conf: 0.390 | Entropy: 1.722
SNR  -10 dB | Acc: 44.62% | Conf: 0.484 | Entropy: 1.454
SNR   -8 dB | Acc: 57.83% | Conf: 0.592 | Entropy: 1.150
SNR   -6 dB | Acc: 70.32% | Conf: 0.689 | Entropy: 0.878
SNR   -4 dB | Acc: 79.41% | Conf: 0.758 | Entropy: 0.685
SNR   -2 dB | Acc: 85.53% | Conf: 0.808 | Entropy: 0.555
SNR    0 dB | Acc: 89.07% | Conf: 0.841 | Entropy: 0.486
SNR    2 dB | Acc: 91.49% | Conf: 0.867 | Entropy: 0.442
SNR    4 dB | Acc: 95.09% | Conf: 0.894 | Entropy: 0.395
SNR    6 dB | Acc: 96.49% | Conf: 0.914 | Entropy: 0.355
SNR    8 dB | Acc: 97.75% | Conf: 0.930 | Entropy: 0.318
SNR   10 dB | Acc: 98.05% | Conf: 0.937 | Entropy: 0.298
SNR   12 dB | Acc: 98.30% | Conf: 0.942 | Entropy: 0.288
SNR   14 dB | Acc: 98.30% | Conf: 0.946 | Entropy: 0.275
SNR   16 dB | Acc: 97.94% | Conf: 0.947 | Entropy: 0.276
SNR   18 dB | Acc: 98.25% | Conf: 0.946 | Entropy: 0.276
SNR   20 dB | Acc: 98.00% | Conf: 0.946 | Entropy: 0.277
SNR   22 dB | Acc: 98.45% | Conf: 0.948 | Entropy: 0.272
SNR   24 dB | Acc: 98.50% | Conf: 0.947 | Entropy: 0.275
SNR   26 dB | Acc: 98.35% | Conf: 0.948 | Entropy: 0.272
SNR   28 dB | Acc: 98.60% | Conf: 0.951 | Entropy: 0.264
SNR   30 dB | Acc: 98.00% | Conf: 0.947 | Entropy: 0.276
```

```
bin_edges = [0.0, 0.2, 0.4, 0.6, 0.8, 1.01]

print("Conf range    n       Accuracy   Mean conf")
for b in range(5):
    lo = bin_edges[b]
    hi = bin_edges[b + 1]
    mask = (all_conf >= lo) & (all_conf < hi)
    n = mask.sum()
    if n > 0:
        print(f"{lo:.1f} - {hi:.1f}   {n:6d}   {correct[mask].mean(
```

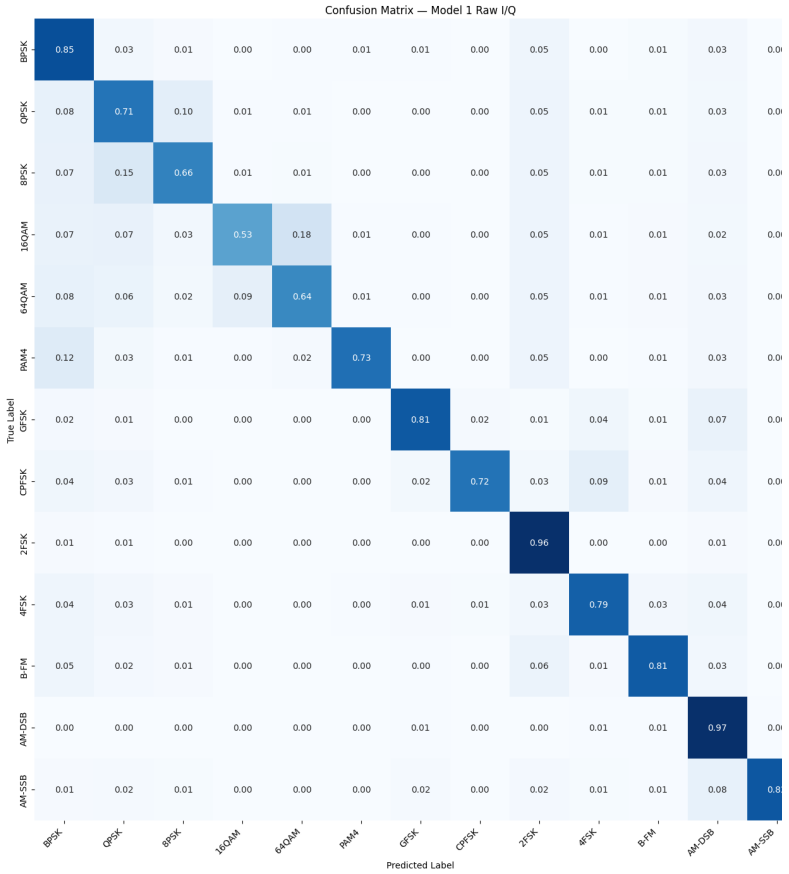
```
Conf range    n       Accuracy   Mean conf
0.0 - 0.2     4945    10.94%    0.145
0.2 - 0.4     5070    23.73%    0.287
0.4 - 0.6     4347    48.03%    0.502
0.6 - 0.8     3391    70.86%    0.701
0.8 - 1.0    34164    98.62%    0.957
```

CONFUSION OF THE MATRIX BECAUSE THE MATRIX IS CONFUSING

```
# Confusion Matrix

cm = confusion_matrix(all_labels, all_preds)
cm_norm = cm.astype('float') / cm.sum(axis=1, keepdims=True)

plt.figure(figsize=(16, 14))
sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues',
            xticklabels=MOD_LABELS, yticklabels=MOD_LABELS)
plt.title("Confusion Matrix — Model 1 Raw I/Q")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

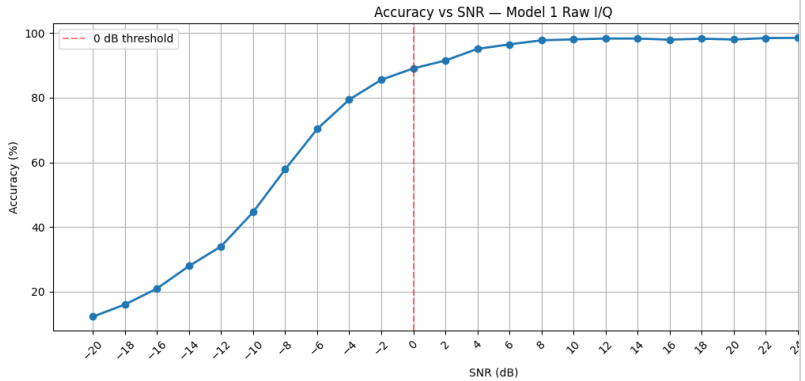


```
# Accuracy vs SNR Plot

# x = snr and y is acc -> depend and independent stuff

snrs_sorted = sorted(snr_accs.keys())
accs = [snr_accs[s]*100 for s in snrs_sorted]

plt.figure(figsize=(12, 5))
plt.plot(snrs_sorted, accs, marker='o', linewidth=2)
plt.axvline(x=0, color='red', linestyle='--', alpha=0.5, label='0 d
plt.title("Accuracy vs SNR — Model 1 Raw I/Q")
plt.xlabel("SNR (dB)")
plt.ylabel("Accuracy (%)")
plt.xticks(snrs_sorted, rotation=45)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



```
    np.savez(os.path.join(save_dir, "iq_v2_eval.npz"),
             preds = all_preds,
             labels = all_labels,
             snrs = all_snrs,
             conf = all_conf,
             true_prob = all_true_prob,
             entropy = all_entropy,
             probs = all_probs)

    print("Saved.", all_probs.shape)
```